USER_MANUAL

Helix Cluster OS — Operator User Manual

Field	Value
Audience	Operators / SREs running Helix Cluster OS locally or on a host they control
Scope	Build, configure, bring services up, run DB migrations, observe health, scan dependencies, host-safety
Status	Living document — verified against the repository at the commit it was written for

This manual documents **only what the code in this repository actually does today.** Where a capability requires infrastructure that is not part of this checkout (a real container runtime, a running PostgreSQL, a deployed multi-node fleet, an SBC/cloud node), that is stated explicitly. Per CLAUDE-1 (anti-bluff) a manual that overstates capability is a forbidden PASS-bluff; per CLAUDE-3 it must stay in sync with the code it describes.

> **Honesty notes you must read first**
>
> - The repository is a Go workspace (`go.work`) with many control-plane command binaries under `cmd/`. Each `cmd/<name>/main.go` is a real `package main`. The list is enumerated below.
> - The default infrastructure orchestrator (`helix_infra`) runs in **simulation mode** out of the box — see §5.3. A real container orchestrator (rootless podman, auto-detected) exists in `pkg/infra/container_orchestrator.go` but is not the default wiring. Treat `make dev` / `helix_infra up` output as simulated unless you have wired the real orchestrator and have a container runtime present.
> - The full distributed multi-node flow is **not validated end-to-end in this checkout** (requires a deployed cluster). Single-process binaries and their HTTP/gRPC surfaces are what you can exercise locally.

---

1. Prerequisites

Component	Version / requirement	Why	Source of truth
Go toolchain	`go 1.25.0` directive, `toolchain go1.26.4` pinned	Build every Go binary + run tests	`go.mod`, `go.work`
Container runtime (rootless)	podman (preferred) / docker / nerdctl, **rootless**	Real container orchestration + integration tests; auto-detected via `runtime.AutoDetect`	`pkg/infra/infra_container_integration_test`
PostgreSQL	16+ reachable on `localhost:5432` (dev default)	Schema migrations target	`scripts/run-migrations.sh`, `migrations/postgresql/`
`golang-migrate` `migrate` CLI	latest, on `PATH` or `~/go/bin`	Migration runner backend	`scripts/run-migrations.sh`
`cyclonedx-gomod`	latest, on `PATH`	SBOM generation (`make sbom`)	`scripts/gen-sbom.sh`
`govulncheck`	latest, on `PATH`	Vulnerability scan (`make deps-update`)	`scripts/deps-update.sh`
Node.js 20+	optional	Web UI dev server only (`web/`)	`README.md`
SQLite 3	optional	Work-item registry at `data/hxc_registry.db`	`README.md`

Install the Go-based tools without elevated privileges (the npm/go prefixes must be user-writable):

```
go install -tags 'postgres' github.com/golang-migrate/migrate/v4/
        cmd/migrate@latest
go install github.com/CycloneDX/cyclonedx-gomod/cmd/cyclonedx-
        gomod@latest
go install golang.org/x/vuln/cmd/govulncheck@latest
# Ensure $(go env GOPATH)/bin (usually ~/go/bin) is on PATH.
```

1.1 No-sudo / rootless rule (hard constraint)

This project forbids `sudo` / `su` and any elevated-privilege workflow. All container work uses **rootless** podman/docker (auto-detected by `pkg/runtime`). If a step seems to "need root", the correct fix is a rootless container or a user-writable tool prefix — never `sudo`. This is a release-blocking constraint, not a style preference.

---

2. Building

2.1 Recommended build path (verified)

The README's canonical build is a whole-workspace Go build:

```
./scripts/setup.sh
        # initialise submodules and toolchains (run once)
./scripts/build.sh     # go build ./...   — builds every Go
        package/binary in the workspace
./scripts/test.sh      # go test ./...
./scripts/lint.sh      # go vet + linters
./scripts/format.sh    # gofmt
```

To build a single control-plane binary into `bin/`:

```
go build -o bin/helixd          ./cmd/helixd
go build -o bin/helix-gateway   ./cmd/helix-gateway
go build -o bin/helix-health    ./cmd/helix-health
# …and so on for any cmd/<name> below.
```

2.2 `make build` — known caveat (do not rely on it as-is)

The `build` target in the root `Makefile` is:

```
build: ## Build all binaries
    go build -o bin/helix-cluster ./cmd/helix-cluster
    zig build
    cmake -B build -S . && cmake --build build
```

`./cmd/helix-cluster` does not exist in this checkout — `go build ./cmd/helix-cluster` fails with `directory not found`. The `zig build` and `cmake` steps depend on a Zig/CMake toolchain and project files that are out of scope for a pure-Go operator. **For building Go binaries use `./scripts/build.sh` (`go build ./...`) or per-binary `go build ./cmd/<name>` as in §2.1.** This caveat is recorded honestly rather than presenting `make build` as a working all-in-one.

2.3 The control-plane command binaries (real, each has `main.go`)

`cmd/helixd`	`cmd/helix-gateway`	`cmd/helix-health`
`cmd/helix-scheduler`	`cmd/helix-session`	`cmd/helix-security`
`cmd/helix-policy`	`cmd/helix-advisory`	`cmd/helix-build`
`cmd/helix-node`	`cmd/helix-llm`	`cmd/helix-snapshot`
`cmd/helix-setup`	`cmd/helix-agent`	`cmd/helix-test`
`cmd/helix_infra`	`cmd/gpu-pool-manager`	`cmd/e2ee-proxy`
`cmd/dst-sim`	`cmd/htmux`	`cmd/hxc-registry`
`cmd/helixctl`		

> `helixctl` (`cmd/helixctl`) is the operator-facing CLI. Its `build` command group is a thin gRPC client of the `helix-build` service (`BuildService`): `helixctl build submit|status|logs|cancel` each perform a real RPC against a running build service at `--addr` (default `localhost:50051`,

> overridable by the `HELIX_BUILD_ADDR` env var). It is built into `bin/helixctl` by `make build` along with the per-service binaries. Other operator interactions continue to happen through the per-service binaries and their HTTP/gRPC surfaces described below.

---

3. Configuration

Configuration is environment-variable driven. There is no single global config file; each binary reads its own variables with localhost defaults.

3.1 `helixd` (control-plane daemon) — `cmd/helixd/main.go`

Variable	Default	Meaning
`HELIXD_PORT`	`8081`	TCP port for the `/status` HTTP surface (`0` = ephemeral)
`HELIXD_HOST`	all interfaces	Bind host
`HELIXD_ETCD_ADDR`	`localhost:2379`	etcd dependency address probed for `/status`
`HELIXD_POSTGRES_ADDR`	`localhost:5432`	PostgreSQL dependency address probed
`HELIXD_REDIS_ADDR`	`localhost:6379`	Redis dependency address probed

`helixd` does not implement business logic itself; it probes its backing subservices (etcd, postgres, redis) over TCP and aggregates their reachability into `/status`. Invalid `HELIXD_PORT` fails fast at startup.

3.2 `helix-gateway` (edge HTTP reverse proxy) — `cmd/helix-gateway/main.go`

Variable	Default	Meaning
`HELIX_GATEWAY_PORT`	`8080`	TCP port to listen on (must be 1–65535)
`HELIX_GATEWAY_HOST`	all interfaces	Bind host

3.3 Database (`make migrate-*` / `scripts/run-migrations.sh`)

`DATABASE_URL` is honored when set; otherwise it is assembled from the `DB_*` parts. No secret is hardcoded — the dev password is `helix` for local development only.

Variable	Default	Meaning
`DATABASE_URL`	`postgres://helix:helix@localhost:5432/`	Full DSN; overrides the parts below

Variable	Default	Meaning
	`helix_cluster?sslmode=disable`	
`DB_HOST`	`localhost`	Host
`DB_PORT`	`5432`	Port
`DB_NAME`	`helix_cluster`	Database
`DB_USER`	`helix`	User
`DB_PASS`	`helix`	Password (dev only — supply a real secret via env in any shared environment)
`DB_SSLMODE`	`disable`	libpq sslmode

3.4 Remote container distribution — `containers/.env` (`Containers/.env.example` template)

The `containers` submodule loads `CONTAINERS_REMOTE_*` env vars (via `containers/pkg/envconfig`). When `CONTAINERS_REMOTE_ENABLED=false` (the default) distribution is local-only. Copy the example template and edit:

```
cp Containers/.env.example Containers/.env   # then edit values
```

Selected variables (full annotated set in `Containers/.env.example`):

Variable	Default	Meaning
`CONTAINERS_REMOTE_ENABLED`	`false`	Master switch for remote distribution
`CONTAINERS_REMOTE_DEFAULT_SSH_USER`	`deploy`	Default SSH user for remote hosts
`CONTAINERS_REMOTE_DEFAULT_SSH_KEY`	`~/.ssh/id_rsa`	Default SSH key
`CONTAINERS_REMOTE_DEFAULT_RUNTIME`	`docker`	Remote runtime (`docker`/`podman`)
`CONTAINERS_REMOTE_SCHEDULER`	`resource_aware`	Placement strategy (`resource_aware`/`round_robin`/`affinity`/`spread`/`bin_pack`)
`CONTAINERS_REMOTE_PORT_RANGE_START` / `_END`	`20000` / `30000`	Local SSH-tunnel port range
`CONTAINERS_REMOTE_HOST_N_NAME` / `_ADDRESS` / `_PORT` / `_USER` / `_KEY` / `_RUNTIME` / `_LABELS`	—	Per-host definitions, `N = 1..` (loader stops at the first absent `_NAME`)

> `.env` and `.env.*` are git-ignored and MUST NOT contain real secrets in tracked files. Only the `.env.example` placeholder template is tracked.

4. Running services

Each binary is a self-contained process. Build it (§2.1), set its env vars (§3), and run it. A binary serves until it receives `SIGINT`/`SIGTERM`, then shuts down gracefully with a bounded timeout.

4.1 helixd (control plane)

```
go build -o bin/helixd ./cmd/helixd
HELIXD_PORT=8081 ./bin/helixd
# Verify the aggregated status surface:
curl -fsS http://localhost:8081/status | jq .
```

`/status` returns `{version, status:"healthy", services: {etcd,postgres,redis: reachable|unreachable}, timestamp}`. `status:"healthy"` means *the daemon itself is serving*; per-subservice reachability is in `services`.

4.2 helix-gateway (edge proxy)

```
go build -o bin/helix-gateway ./cmd/helix-gateway
HELIX_GATEWAY_PORT=8080 ./bin/helix-gateway
```

The gateway is a path-prefixed HTTP reverse proxy to backend services. It stamps an `X-Helix-Gateway` marker header on proxied responses and exposes Prometheus `/metrics`. Routed API paths include `/v1/sessions`, `/v1/pool/utilization`, and `/api/v1/scheduler/…` (the last is scope-enforced; denials carry an `X-Helix-Deny-Reason` header). Backends must be running for proxied routes to return non-error responses.

4.3 helix-health (health aggregation)

```
go build -o bin/helix-health ./cmd/helix-health
./bin/helix-health
```

Serves the `helix.v1.HealthService` gRPC API plus an HTTP surface exposing `/health`, `/livez`, `/readyz`, and `/check/<service>` (see §6).

4.4 Other control-plane binaries

`helix-scheduler`, `helix-session`, `helix-security`, `helix-policy`, `helix-advisory`, `helix-build`, `helix-node`, `helix-llm`, `helix-snapshot`, `helix-agent`, `gpu-pool-manager`, `e2ee-proxy` each build the same way (`go build ./cmd/<name>`) and read their own env. Inspect the binary's `--help` / source `LoadConfig` for its exact flags and variables before relying on them.

---

5. Infrastructure orchestration (`helix_infra`)

`helix_infra` is the dev-infrastructure manager (PostgreSQL, Redis, etcd, NATS/ Kafka, Prometheus/ Grafana/Jaeger, Vault, and QEMU VM-node simulation). The Makefile wraps it:

Make target	Underlying command	Effect
`make dev`	`go run ./cmd/ helix_infra up --wait --timeout 300s`	Boot the infra stack
`make dev-down`	`go run ./cmd/ helix_infra down`	Stop the stack
`make dev-status`	`go run ./cmd/ helix_infra status`	Show per-service status table
`make dev-logs service=<name>`	`go run ./cmd/ helix_infra logs <name>`	Stream a service's logs

Direct invocation (subcommands: `up`, `down`, `status`, `health`, `logs`, `scale`, `vm`, `version`):

```
go run ./cmd/helix_infra up --wait --timeout 300s
go run ./cmd/helix_infra status
go run ./cmd/helix_infra health --json
go run ./cmd/helix_infra down
```

`up` accepts an optional `--config <file>` (JSON overlaying the built-in `DefaultConfig`; a missing or malformed path is a hard error, never silently ignored).

5.3 Simulation vs real container orchestration

The orchestrator returned by the default wiring is a simulation. Its own guard test asserts `Simulated()==true` "until a real Orchestrator is wired in" (`pkg/ infra/infra_integration_test.go`). So `make dev` / `helix_infra up` reports services "healthy" against a simulated backend, **not** against real containers, unless you have wired the real orchestrator.

A **real** container orchestrator exists at `pkg/infra/container_orchestrator.go` (`Simulated()==false`); it uses a rootless container runtime auto-detected by `runtime.AutoDetect` (podman preferred). Its integration tests skip with an explicit `SKIP-OK` reason when no CLI container runtime is present, and assert hard (no skip) when podman is available. **Treat real container boot as requiring a present rootless runtime; it is not exercised by the default path.**

> The legacy `make dev-compose` / `dev-compose-down` targets call `docker compose` directly. Per the project's no-manual-container-commands rule, prefer the orchestrator path over hand-running compose.

---

6. Health and observability

Surface	Exposed by	Endpoints	Notes
Daemon status	`helixd`	`GET /status`	Aggregated daemon + subservice reachability JSON
Health service	`helix-health`	`GET /health`, `/livez`, `/readyz`, `/check/<service>`; gRPC `helix.v1.HealthService`	Liveness/readiness + per-service check
Metrics	`helix-gateway`, `helixd` (and other services via `pkg/metrics`)	`GET /metrics`	Prometheus exposition mounted via `metrics.Mount`

```
curl -fsS http://localhost:8081/status        # helixd
curl -fsS http://localhost:8080/metrics        # gateway
        Prometheus metrics
curl -fsS http://localhost:<health-http>/livez # helix-health
        liveness
```

> The `docs/guides/operations.md` guide describes a Prometheus stack on `:9090/metrics`, Grafana dashboards, `/healthz`/`readyz` probes and Kubernetes deployment. Those are **target operational patterns / deployment options**, not all wired into the binaries in this checkout. The authoritative per-binary endpoints are the ones in the table above (`/status`, `/metrics`, `/health`+`/livez`+`/readyz`+`/check/<service>`). Verify the actual port for `helix-health` from its `LoadConfig` before scripting probes against it.

7. Database migrations

Migrations are golang-migrate SQL files in `migrations/postgresql/` (15 sequential `*.up.sql` / `*.down.sql` pairs, e.g. `001_create_nodes`, `002_create_gpu_devices`, `003_create_sessions`, …). The runner is `scripts/run-migrations.sh`, requiring the `migrate` CLI (§1).

Command	Effect
`make migrate-up`	`run-migrations.sh up` — apply all pending migrations (honors `DATABASE_URL`)
`make migrate-down`	`run-migrations.sh down 1` — roll back one migration

Direct use of the runner exposes more subcommands:

```
./scripts/run-migrations.sh up [N]      # apply all, or N
./scripts/run-migrations.sh down [N]    # roll back 1, or N
./scripts/run-migrations.sh version     # current schema version
./scripts/run-migrations.sh status      # dir, DSN, version, file
        list
./scripts/run-migrations.sh validate    # up -> down 15 -> up
        (round-trip check)
./scripts/run-migrations.sh create NAME # scaffold a new up/down
        pair
./scripts/run-migrations.sh force V
        # force version (use with caution)
```

Point at a real PostgreSQL via env (no hardcoded secret):

```
DB_HOST=db.internal DB_NAME=helix_cluster DB_USER=helix
        DB_PASS="$PGPASSWORD" \
  ./scripts/run-migrations.sh up
# or:
DATABASE_URL='postgres://helix:***@db.internal:5432/helix_cluster?
        sslmode=require' make migrate-up
```

> **Requires a reachable PostgreSQL.** Migrations run against a live database; there is no embedded DB. `make seed` (`scripts/seed-data.sql`) targets `…/helix?sslmode=disable` and is for local development seeding only.

---

8. SBOM and vulnerability scanning

Command	Underlying script	Effect
`make sbom`	`scripts/gen-sbom.sh`	Generate CycloneDX 1.6 SBOMs into `sbom/`
`make deps-update`	`scripts/deps-update.sh`	Bump deps + `go work sync` + `govulncheck` + refresh SBOMs

8.1 `make sbom`

Emits one CycloneDX JSON per tracked Go module (workspace disabled per-module so the root component is unambiguous):

```
sbom/helixcluster.cdx.json   # main module
sbom/api-v1.cdx.json         # api/v1
sbom/security.cdx.json       # digital.vasic.security
```

Requires `cyclonedx-gomod` on `PATH` (the target prints install instructions and exits non-zero if missing). The `sbom/*.cdx.json` outputs are build derivatives — git-ignored; the script + Make target are the tracked deliverable, and `make sbom` regenerates them.

8.2 `make deps-update` (local, no-CI dependency maintenance)

The project forbids CI/CD pipelines, so there is no dependabot/renovate. `make deps-update` is the operator-run equivalent across modules `.`, `api/v1`, `security`:

1. `go get -u ./...` per module.
2. `go work sync`.
3. `govulncheck ./...` per module — **non-zero exit on reachable advisories** (or if `govulncheck` is missing).
4. Refresh SBOMs (`scripts/gen-sbom.sh`).

```
make deps-update
        # review the govulncheck output before committing bumps
govulncheck ./...           # standalone scan of the current
        module
```

---

9. Host-safety constraints (mandatory)

These are release-blocking invariants, binding on every command, script, and agent action:

1. **No host power-state transitions.** Never run (directly or indirectly) `systemctl`/`loginctl` suspend/hibernate/poweroff/halt/reboot, `pm-suspend`, `shutdown`, or DBus power calls. The host runs mission-critical workloads; auto-suspend has caused historical session loss.
2. **No `sudo` / `su`.** Use rootless containers (the `containers` submodule / `pkg/runtime`) or user-writable tool prefixes. See §1.1.
3. **Memory budget ≤ 60% of host RAM.** Project procedures must not exceed 60% of total system memory; the remaining 40% is reserved for the operator's other workloads. Reduce parallelism / close other workloads rather than raising the cap (there is no override flag).
4. **Resource-bounded tests/challenges.** Run test and challenge workloads at ~30–40% of host resources (`GOMAXPROCS=2`, `nice -n 19`, `ionice -c 3` where applicable). Example: `GOMAXPROCS=2 nice -n 19 go test -short ./...`.
5. **No manual container commands as a workflow.** Bring containers up through the orchestrator / `containers` submodule, not hand-run `docker`/`podman start|stop|rm` or `docker compose up`.
6. **Container hygiene.** Containers must declare explicit memory limits and be cleanly destroyed + rebuilt after any host crash so stale state does not reproduce failures.

If a host is already under memory pressure (`journalctl -k --since "1 hour ago" | grep -c oom-kill` is non-zero), fix the offending workload before stacking new heavy work.

---

10. Quick reference

```
# Build (Go workspace)
./scripts/build.sh                          # go build ./...
        (preferred — NOT `make build`)
```

```
go build -o bin/helixd ./cmd/helixd      # single binary

# Run core surfaces
HELIXD_PORT=8081 ./bin/helixd            # /status on :8081
HELIX_GATEWAY_PORT=8080 ./bin/helix-gateway
        # /metrics + /v1/* on :8080

# Dev infrastructure (simulation by default)
make dev        /  make dev-status  /  make dev-down

# Database migrations (needs a live PostgreSQL + `migrate` CLI)
make migrate-up  /  make migrate-down
./scripts/run-migrations.sh status

# Supply-chain
make sbom        /  make deps-update  /  govulncheck ./...

# Resource-bounded tests
GOMAXPROCS=2 nice -n 19 go test -short ./...
```

Sources verified

This manual was written by reading the following in-repo sources (paths relative to repo root):

- `Makefile` — all targets (`build`, `dev`/`dev-*`, `migrate-up`/`migrate-down`, `seed`, `sbom`, `deps-update`, `docs*`)
- `go.mod`, `go.work` — Go toolchain pin and workspace membership
- `cmd/helixd/main.go` — `HELIXD_*` env, `/status` surface, graceful shutdown
- `cmd/helix-gateway/main.go` — `HELIX_GATEWAY_*` env, `/metrics`, `X-Helix-Gateway`
- `cmd/helix-health/main.go` — gRPC `HealthService` + `/health`,`/livez`,`/readyz`,`/check/<service>`
- `cmd/helix_infra/{main,up,run,shared,health}.go` — orchestrator subcommands; simulation-by-default
- `pkg/infra/infra_integration_test.go`, `pkg/infra/container_orchestrator.go`, `pkg/infra/infra_container_integration_test.go` — `Simulated()` default + real rootless-podman path
- `internal/gateway/*_integration_test.go` — routed API paths and scope/deny header
- `scripts/run-migrations.sh`, `migrations/postgresql/` (15 up/down pairs) — migration runner + DB env
- `scripts/gen-sbom.sh`, `scripts/deps-update.sh` — SBOM + vuln-scan flows
- `Containers/.env.example`, `containers/pkg/envconfig` — remote-distribution env vars
- `README.md`, `docs/guides/getting-started.md`, `docs/guides/operations.md` — cross-checked; their `helixctl`, `:9090/metrics`, K8s, and `make build` claims are corrected/qualified above against the actual binaries and targets present in this checkout.